Asistente con IA: cerebro coordinador y agentes especialistas

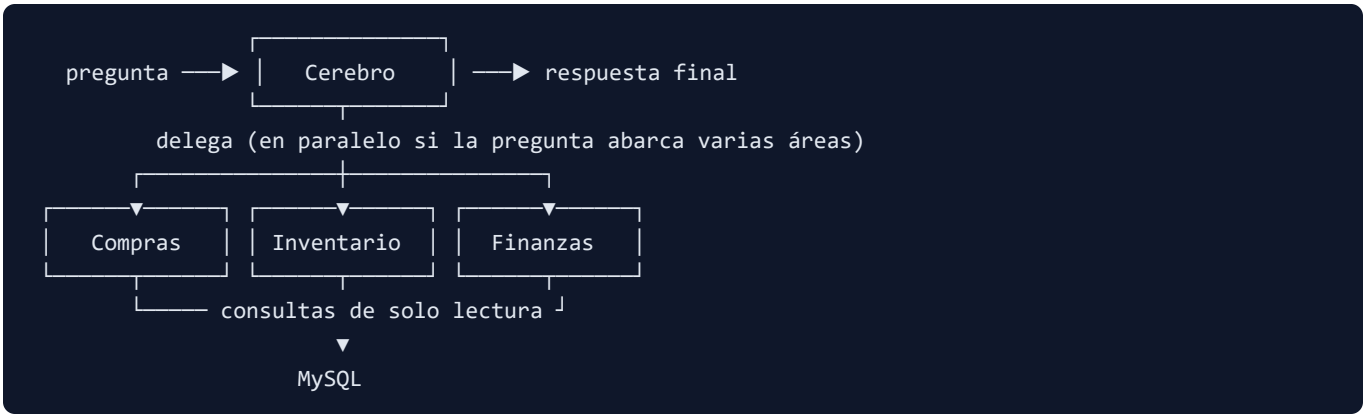
Sistema de Órdenes de Compra · Diseño, decisiones y pruebas del asistente multiagente.

1. Qué problema resuelve

Las pantallas del sistema muestran cada área por separado: órdenes, inventario, facturas. Para responder una pregunta de negocio como "*¿qué debo reponer, qué tengo por pagar y qué falta aprobar?*", una persona tendría que abrir tres pantallas y cruzar los datos a mano.

El asistente responde ese tipo de preguntas en lenguaje natural, con los datos reales de MySQL y en segundos.

2. Arquitectura



Pieza	Qué hace	Herramientas
Cerebro	Recibe la pregunta, decide a qué agentes delegarla, redacta la tarea de cada uno y combina sus informes. No accede a la base de datos.	`delegar_compras`, `delegar_inventario`, `delegar_finanzas`
Agente de Compras	Órdenes de compra, requisiciones, cotizaciones y proveedores.	`listar_ordenes`, `listar_requisiciones`, `listar_cotizaciones`, `listar_proveedores`
Agente de Inventario	Existencias por bodega, faltantes frente al mínimo, bodegas y recepciones.	`listar_existencias`, `listar_bodegas`, `listar_recepciones`
Agente de Finanzas	Facturas, saldos, vencimientos, pagos y gasto frente al presupuesto.	`listar_facturas`, `listar_pagos`, `gasto_por_departamento`

Cómo funciona por dentro. Cada pieza es una conversación con el modelo **Claude Opus 5.5** de Anthropic, a través de la API de mensajes con *uso de herramientas* (*tool use*):

1. El modelo responde pidiendo usar una herramienta.
2. El servidor la ejecuta y le devuelve el resultado.
3. El ciclo se repite hasta que el modelo da su respuesta.

Para el cerebro, "usar una herramienta" significa poner a trabajar a un agente; para cada agente, significa consultar MySQL. Cuando el cerebro pide varios agentes en la misma respuesta, el servidor los ejecuta **en paralelo** (`Promise.all`).

Archivo	Contenido
`backend/src/asistente/agentes.js`	Cerebro, agentes, ciclo de herramientas, consumo
`backend/src/asistente/datos.js`	Las 10 consultas de solo lectura
`backend/src/controllers/asistenteController.js`	Endpoint y transmisión en vivo
`frontend/src/pages/AsistentePage.jsx`	Chat
`frontend/src/components/MapaAgentes.jsx`	Mapa animado

3. Transmisión en vivo y mapa animado

`POST /api/asistente` no espera a tener la respuesta completa: envía una línea JSON por cada paso, en formato **NDJSON**. Los pasos son:

- `cerebro_piensa`
- `delegar`
- `herramienta` y `herramienta_lista` (el agente consulta MySQL y recibe los datos)
- `agente_responde`
- `respuesta` y `consumo`

La pantalla traduce cada paso en movimiento: el agente baja hacia MySQL cuando consulta, sube hacia el cerebro cuando entrega su informe, y un punto de luz viaja por la conexión. Además se va llenando un registro de actividad.

Se eligió NDJSON sobre *fetch* en lugar de `EventSource` porque `EventSource` no permite enviar el encabezado `Authorization: Bearer <token>` que exige la API.

4. Decisiones de diseño

Decisión	Motivo
Varios agentes especializados en lugar de uno solo	Cada agente ve solo las herramientas de su área (3 o 4 en lugar de 10), así elige mejor y sus instrucciones son más cortas. También permite trabajar en paralelo.
Herramientas de solo lectura	Un modelo de lenguaje puede equivocarse o ser manipulado con texto malicioso dentro de los datos (*prompt injection*). Si solo puede leer, el peor caso es una respuesta incorrecta, nunca datos dañados. Crear o cambiar registros sigue haciéndose en las pantallas, con sus validaciones.
Consultas predefinidas, no SQL libre	El modelo no escribe SQL: elige entre 10 consultas ya escritas y revisadas, con filtros validados (por ejemplo, `estado` solo acepta valores de una lista). Así no hay riesgo de inyección SQL ni de consultas costosas.
Máximo de pasos	El cerebro y cada agente tienen un tope de 6 vueltas, para que un error del modelo no genere un ciclo infinito de llamadas (y de costo).
Datos como información, no como órdenes	Las instrucciones de cada pieza indican que el contenido de la base es información y nunca debe obedecerse como una orden.
Esfuerzo bajo en los agentes y medio en el cerebro	Los agentes hacen tareas acotadas (consultar y resumir); el cerebro razona más al combinar. Así se contiene el costo sin perder calidad.

Decisión	Motivo
Respaldo automático (`fallbacks: "default"`)	Si el modelo declina una consulta por sus filtros de seguridad, la API la reintenta con otro modelo en la misma llamada.
La clave fuera del código	`ANTHROPIC_API_KEY` vive en `backend/.env`, que no se sube al repositorio. Sin clave, el resto del sistema funciona igual y la pantalla explica cómo configurarla.

5. Control del costo

Cada pregunta cuesta dinero en la cuenta de Anthropic. Por eso:

- **Límite de uso:** 10 preguntas por minuto por usuario. Si se supera, la API responde **429** con el tiempo de espera. Se configura con `LIMITE_ASISTENTE`.
- **Consumo visible:** cada respuesta muestra cuántas llamadas al modelo hizo, los tokens de entrada y de salida, y el costo estimado. Se usan los precios de Claude Opus 5.5: US$4 por millón de tokens de entrada y US$20 por millón de salida.
- **Consumo medido en pruebas reales:**

Pregunta	Agentes	Llamadas	Tokens (entrada / salida)	Costo
¿Qué debo reponer, qué facturas tengo por pagar y qué órdenes faltan por aprobar?	3	8	10 395 / 2 022	≈ US$0.08
Compara en una tabla las cotizaciones vigentes	1	4	6 881 / 1 849	≈ US$0.07

6. Seguridad

- Las rutas del asistente están detrás del mismo middleware JWT que el resto de la API: sin token responden 401.
- La pregunta (máximo 1000 caracteres) y el historial (máximo 20 mensajes, solo los roles `usuario` y `asistente`) se validan. Cualquier otro campo se rechaza con 400.
- La respuesta del modelo se muestra sin HTML crudo: el componente `TextoMarkdown` convierte el Markdown (negritas, listas, tablas) en elementos de React, que siempre escapan el texto.
- El login también tiene límite (30 intentos cada 15 minutos por IP) para frenar ataques de fuerza bruta.

7. Pruebas

Tipo	Qué se comprueba
`npm test` (8 pruebas, `backend/test/asistente.test.js`)	Con un **cliente simulado** (no llama a la API ni gasta dinero): delegación en paralelo, que cada agente reciba solo sus herramientas, el cálculo del consumo, que **la base de datos no cambie** después de una pregunta, el manejo de una herramienta inexistente, de un rechazo del modelo y del tope de pasos, y el límite de uso (429).
Postman (carpeta *Asistente IA*)	Estado del asistente, sin token (401), pregunta vacía, rol inválido y campo extra (400).
Pruebas reales	Preguntas con la clave configurada: respuestas correctas contra los datos de la base, con tablas y consumo visibles (capturas en `docs/evidencias/`).

8. Limitaciones y mejoras posibles

- **Solo consulta.** Una mejora sería un agente que proponga acciones (por ejemplo, crear una requisición para lo que está bajo el mínimo) y las ejecute solo si el usuario las confirma.
- **La respuesta final aparece completa** al terminar. Se podría transmitir palabra por palabra.
- **El límite de uso se guarda en memoria:** si hubiera varios servidores, habría que moverlo a un almacenamiento compartido, como Redis.
- **El costo es una estimación** con los precios publicados; el consumo exacto se ve en la consola de Anthropic.

Fuentes

1. Anthropic. *Tool use with Claude*. https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview
2. Anthropic. *Messages API*. https://platform.claude.com/docs/en/api/messages
3. Anthropic. *Building effective agents*. https://www.anthropic.com/engineering/building-effective-agents
4. OWASP. *LLM Prompt Injection Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html